



计算机应用
Journal of Computer Applications
ISSN 1001-9081, CN 51-1307/TP

《计算机应用》网络首发论文

题目: 具备可解释性的软件缺陷预测数据集构建方法
作者: 边赟, 王海全, 陈义, 崔喆
收稿日期: 2025-08-27
网络首发日期: 2025-11-17
引用格式: 边赟, 王海全, 陈义, 崔喆. 具备可解释性的软件缺陷预测数据集构建方法 [J/OL]. 计算机应用. <https://link.cnki.net/urlid/51.1307.TP.20251114.1526.006>



网络首发: 在编辑部工作流程中, 稿件从录用到出版要经历录用定稿、排版定稿、整期汇编定稿等阶段。录用定稿指内容已经确定, 且通过同行评议、主编终审同意刊用的稿件。排版定稿指录用定稿按照期刊特定版式 (包括网络呈现版式) 排版后的稿件, 可暂不确定出版年、卷、期和页码。整期汇编定稿指出版年、卷、期、页码均已确定的印刷或数字出版的整期汇编稿件。录用定稿网络首发稿件内容必须符合《出版管理条例》和《期刊出版管理规定》的有关规定; 学术研究成果具有创新性、科学性和先进性, 符合编辑部对刊文的录用要求, 不存在学术不端行为及其他侵权行为; 稿件内容应基本符合国家有关书刊编辑、出版的技术标准, 正确使用和统一规范语言文字、符号、数字、外文字母、法定计量单位及地图标注等。为确保录用定稿网络首发的严肃性, 录用定稿一经发布, 不得修改论文题目、作者、机构名称和学术内容, 只可基于编辑规范进行少量文字的修改。

出版确认: 纸质期刊编辑部通过与《中国学术期刊 (光盘版)》电子杂志社有限公司签约, 在《中国学术期刊 (网络版)》出版传播平台上创办与纸质期刊内容一致的网络版, 以单篇或整期出版形式, 在印刷出版之前刊发论文的录用定稿、排版定稿、整期汇编定稿。因为《中国学术期刊 (网络版)》是国家新闻出版广电总局批准的网络连续型出版物 (ISSN 2096-4188, CN 11-6037/Z), 所以签约期刊的网络版上网络首发论文视为正式出版。

具备可解释性的软件缺陷预测数据集构建方法

边 赞^{1,2}, 王海全^{1,2}, 陈 义^{1,2}, 崔 喆^{1,2*}

(1. 中国科学院 成都计算机应用研究所, 成都 610213; 2. 中国科学院大学 计算机科学与技术学院, 北京 100049)

(* 通信作者电子邮箱 cuizhe@casic.com.cn)

摘要:针对软件缺陷预测缺乏缺陷定位、缺陷解释及修复建议等可解释性信息,导致预测结果难以在实际开发中应用的问题,提出一种基于上下文工程与大语言模型(LLMs)的、具有可解释性的软件缺陷预测数据集构建方法,并发布了首个配套的多编程语言的软件缺陷预测数据集HandPick。首先,基于软件工程原则和缺陷先验知识,设计了TriCogVuln-LLM方法,引导LLMs依次完成功能描述生成、通用缺陷枚举(CWE)缺陷预测和缺陷修复建议生成。其次,设计了共识投票机制,构建了缺陷预测的最佳生成模型池,进一步提升了生成数据的质量与多样性。最后,利用共识驱动的自动化数据生成,构建出包含四种主流编程语言的软件缺陷预测数据集——HandPick。下游任务验证表明,与基线模型相比,采用HandPick数据集微调后的Qwen2.5-14B-HandPick模型在独立公开测试集的表现显著提升,其精确率、召回率、F1分数与准确率分别提升了19.29、21.26、24.11与18.30个百分点,证明TriCogVuln-LLM方法构建的HandPick数据集能够显著提升模型的缺陷识别和分析能力,有望辅助开发人员更好地修复软件缺陷。

关键词:可解释性;软件缺陷预测;上下文工程;大语言模型;通用缺陷枚举

中图分类号:TP391.1 文献标志码:A

Construction of software defect prediction dataset with explainability

BIAN Yun^{1,2}, WANG Haiquan^{1,2}, CHEN Yi^{1,2}, CUI Zhe^{1,2*}

(1. Chengdu Institute of Computer Application, Chinese Academy of Sciences, Chengdu Sichuan 610213, China;

2. School of Computer Science and Technology, University of Chinese Academy of Sciences, Beijing 100049, China)

Abstract: Software defect prediction often lacks explainable information, such as defect localization, explanation, and repair suggestions. This limitation makes the prediction results difficult to apply in actual development. To address this problem, this paper proposes an explainable approach for constructing a software defect prediction dataset based on context engineering and large language models (LLMs). In addition, it presents HandPick, the first multi-programming-language dataset for software defect prediction. First, the TriCogVuln-LLM method was designed based on software engineering principles and prior defect knowledge, guiding LLMs to sequentially generate function descriptions, Common Weakness Enumeration (CWE) defect predictions, and repair suggestions. Next, a consensus voting mechanism was employed to form an optimal ensemble of generative models for defect prediction, thereby improving the quality and diversity of the generated data. Finally, the HandPick dataset was constructed through consensus-driven automated data generation, covering code in four mainstream programming languages. Downstream task validation shows that the Qwen2.5-14B-HandPick model, fine-tuned on the HandPick dataset, achieves significant improvements over baseline models on an independent, publicly available test set, with gains of 19.29, 21.26, 24.11, and 18.30 percentage points in precision, recall, F1 score, and accuracy, respectively. This confirms that the HandPick dataset, generated by the TriCogVuln-LLM method, substantially enhances a model's defect identification and analysis capabilities, offering a promising approach to better assist developers in remediating software defects.

Key words: explainability; software defect prediction; context engineering; large language models; common weakness enumeration

0 引言

软件缺陷预测(software defect prediction)是软件工程领域的重要研究方向,其主要目标是在软件开发早期阶段预测潜在缺陷,以优化测试资源的分配,从而降低修复成本、提升软件质量^[1]。软件缺陷不

仅会对软件的功能性产生负面影响,严重的时候会给企业带来巨额的经济损失,甚至有时候会引发人员伤亡。随着开源项目规模持续扩大和自动化开发工具激增,软件缺陷的发生时间和数量变得更加不可控,缺陷预测技术亟需尽早融入整个软件开发生命周期,以尽可能低的成本最大化地提升缺陷发现率、优化测试资源分配、保障软件

收稿日期:2025-08-27;修回日期:2025-11-04;录用日期:2025-11-07。 基金项目:西部之光青年学者[2023]16号。

作者简介:边赞(1988—),女,甘肃酒泉人,高级工程师,博士研究生,主要研究方向:自然语言处理、大语言模型;王海全(1999—),男,河南焦作人,博士研究生,主要研究方向:自然语言处理、大语言模型;陈义(2001—),男,河南郑州人,硕士研究生,主要研究方向:自然语言处理、大语言模型;崔喆(1970—),男,四川巴中人,研究员,博士,主要研究方向:自然语言处理、大语言模型。

质量。

尽管现有数据集研究工作在软件缺陷预测取得了一定的进展,但大多属于判断缺陷有无的二分类任务或缺陷类型的多分类任务,且多分类数据集往往缺乏标准的、统一的缺陷分类标准。随着软件开发复杂性的日益提高,数据集可解释性的缺失,进一步提升了软件缺陷预测模型在软件质量保障的应用难度。当前主流的软件缺陷预测数据集根据其待预测数据分为两类:一类是数据特征数据集,如 OSPTTrack 等^[2],其适用模型为传统的机器学习模型,泛化能力受限;另外一类待预测数据为代码,如 Defectors^[3]、CodeXGLUE^[4]等,适用模型为具有良好语言理解能力的模型,具有很好的泛化性。但仅具备语言理解能力的模型通常仅能判断代码是否可能存在缺陷,无法告知开发人员缺陷定位、缺陷类型、缺陷产生的原因和修复建议。这种使用自然语言进行缺陷预测解释能力的缺失,严重阻碍了预测结果在实际软件开发工作中的应用,降低了预测结果的可信度与实用价值^[5]。软件缺陷预测模型的能力取决于其使用的数据集,因此,一个好的软件缺陷预测数据集,不仅具有良好的软件缺陷的分类,还需要具备缺陷的原因等可解释性信息。

通用缺陷枚举(Common Weakness Enumeration, CWE)提供了一种标准化的缺陷分类体系,通过统一的术语和分类方法,帮助开发者更高效地识别和处理软件漏洞。近年来,大语言模型(Large Language Models, LLMs)已成功应用于代码补全、自动生成及测试等多种与代码相关的任务,为解决上述可解释性瓶颈提供了新的契机^[6-8]。理论上,不仅可以利用 LLMs 的代码理解能力预测缺陷,还可以结合其卓越的自然语言生成能力,进一步提供潜在缺陷的精确位置与缺陷原因等解释性信息。在这个理论框架下,研究焦点从传统的以模型为中心转向了以数据为中心,高质量训练数据的供给规模与质量,已成为决定大模型应用水平的核心瓶颈,在同等计算资源下,高质量数据集对模型性能的贡献,远比模型参数规模的增长更为关键。因此,如何系统性地利用 LLMs 构建高质量、细粒度的软件缺陷预测数据集,并以此驱动模型获得可靠且可解释的预测能力,目前仍是一个关键挑战。

针对上述问题,本文提出了一种新颖的软件缺陷预测数据集构建方法,并发布了领域内首个面向大语言模型微调具有可解释性的多编程语言的软件缺陷预测数据集 HandPick,以弥补研究工作的空白。本文的主要贡献包括:

- 1) 设计了 TriCogVuln-LLM (LLM-Enhanced Triple Cognitive Chain for Multilingual Code Vulnerability Mining with CWE Knowledge) 方法,系统性地解决了如何引导 LLMs 生成高质量、具备完整解释链条的软件缺陷数据这一核心难题。
- 2) 本文设计并实现了一套基于“先净化、后共识”原则的共识投票机制,有效提升了生成数据的质量和多样性,解决了单一 LLM 生成数据时固有的系统性偏差问题。
- 3) 提出了一个包含缺陷解释与修改建议的高质量、多编程语言缺陷预测数据集 HandPick,填补了软件缺陷预测领域可解释性数据集的空白,其分析层次从基础的缺陷分类任务,提升至更精确的缺陷定位与分析。

实现代码: <https://github.com/pan-xi/handdpick>, HandPick 数据集: <https://huggingface.co/datasets/pansys/handdpick/tree/main>。

1 相关工作

1.1 软件缺陷预测模型的发展

软件缺陷预测模型的发展可大致分为两个阶段。早期的软件缺陷预测方法依赖于静态代码度量(如 McCabe 圈复杂度)结合传统机器学习模型,如逻辑回归、支持向量机和随机森林等统计模型^[9-10]。这些方法仅在特定项目上有一定效果,但严重依赖于手工设计的特征工程,泛化能力受限。随着深度学习的发展,第二阶段的研究转向了利用神经网络自动学习代码的深层语义和结构特征。例如,研究者们利用基于循环神经网络(Recurrent Neural Network, RNN)或长短期记忆网络(Long Short-Term Memory, LSTM)的模型处理代码序

列^[11],或利用基于抽象语法树(Abstract Syntax Tree, AST)和图神经网络(Graph Neural Network, GNN)的模型来捕捉代码的结构信息。这些深度学习模型显著提升了预测的准确性。

然而,这些模型大多以“黑盒”或“灰盒”形式运行,仅能判断一段代码是否可能存在缺陷,其黑箱特性降低了可解释性,导致开发人员在实际工业场景中无法定位软件缺陷的具体代码片段,存在显著的“产学研”鸿沟,这种可解释性的缺失构成了该领域亟待解决的挑战。

1.2 软件缺陷预测中的可解释性研究

在深度学习中,可解释性通常指的是预测模型以人类可理解的方式解释其决策的能力。当前软件缺陷预测的可解释性的研究未形成共识,本研究认为缺陷预测行为的可解释性应包括缺陷分类、缺陷定位、缺陷原因、修复建议等信息。

但预测模型不具备可解释性也导致了软件缺陷预测行为可解释性的缺失。由于模型本身的可解释性是长期且十分困难的挑战,研究者们将提升软件缺陷预测可解释性的目光从预测模型的可解释性,转向了缺陷预测行为本身的可解释性,从不同角度探索了提升软件缺陷预测可解释性的方法。

一类方法是通过采用如局部可解释性模型诊断解释(Local Interpretable Model-agnostic Explanations, LIME)、SHAP 等事后解释的方法进行局部近似或特征重要性排序来提高结果的可解释性,但它们在不同的数据集或模型设置下可能会产生不一致的结果。例如,LIME 针对同一预测结果所生成的解释,在不同的采样策略下可能存在显著差异,这使其可靠性受到质疑^[12]。此外,该方法使用的特征重要性排序对开发者而言过于抽象,难以直接映射为具体的缺陷原因和修复方案。

另一类方法则聚焦于提供更结构化的解释,以缓解现有事后解释方法的抽象性和难以映射到具体缺陷原因的局限性。CWE 作为一种标准化的缺陷分类体系,通过统一的术语和分类方法,为开发者理解和处理软件缺陷提供了共同语言。在理论层面上,若预测模型能够直接输出与缺陷代码相对应的 CWE 编号,不仅可以显著增强预测结果的可解释性,还可为开发人员提供具体的修复建议。

然而,尽管 CWE 在工业界已得到广泛应用,但其在自动化软件缺陷预测领域仍处于起步阶段,在提升软件缺陷预测的准确性与一致性方面具有巨大潜力^[13-15]。

1.3 大语言模型与上下文工程

大语言模型^[16]在自然语言处理领域的成功,推动了其在代码补全、代码评审等任务中的应用。近期的研究,探索了 LLMs 在应对各种软件工程领域挑战方面的能力^[17-21]。

上下文工程为 LLMs 提供一个最优的“信息场”,为模型提供每次交互中解决任务所需的最完备、最相关的背景知识。通过提供充分且自洽的上下文,避免了信息缺失或不一致导致的推理错误^[25],从而显著提升了模型的推理能力与生成质量。其核心包括两类 workflow: 1) 信息筛选与检索,精确筛选出与当前任务最相关的内容,可以避免冗余或无关信息导致 LLMs 输出质量下降^[26]。2) 上下文结构化,通过特定的格式(如 JSON、XML)、分隔符或元标签来构建结构化的上下文,可以帮助模型更好地区分不同来源和类型的信息。这方面的新兴研究,如“位置工程”,就通过操纵信息在上下文中的位置来提升模型性能^[27]。

上下文工程通常与提示策略结合使用,常见的策略包括把复杂问题分解为多步的思维链、以树状搜索组织的思维树及基于规划的推理流程,这些策略通过引导模型显式地生成中间推理步骤或候选思路,提高模型解决复杂任务的鲁棒性与可解释性。

在代码相关任务中,上下文工程策略已被证明能显著提升模型对代码语义的理解与推理稳定性,使 LLMs 能够克服早期技术的诸多局限性^[19-24]。同时,研究者开始利用 LLMs 处理软件漏洞相关任务,但大多聚焦于现有数据集上的缺陷检测^[21]。

尽管上下文工程在代码任务中已证明其有效性,但系统性地应用该技术,以数据生成的范式来解决可解释性缺陷预测数据的稀缺问题,目前仍是一个有待探索的研究领域。

综上所述,现有研究在软件缺陷预测领域存在三个主要局限性:

- 1) 模型可解释性不足:传统及深度学习模型普遍为“黑盒”,无法提供开发者可直接利用的缺陷定位与分析信息。
- 2) 高质量数据匮乏:可解释性研究因缺乏缺陷定位及结构化解释信息的训练数据集而进展缓慢。
- 3) LLMs应用范式局限:现有工作主要遵循利用已有数据集进行能力评测与模型微调的研究范式,未充分利用大语言模型的能力来生成具备可解释性的高质量数据。

为克服上述局限,本文的工作利用高性能的闭源大模型构建一个模型池,通过上下文工程技术和共识投票机制,实现缺陷预测和修复建议的自动化数据生成。这种方法不仅提升了缺陷预测数据的多样性与准确性,也为代码分析作为一项结构化信息处理任务开辟了新的路径。

2 研究方法

本文的数据集构建方法主要分为3个阶段,分别是:数据收集及数据预处理、最佳生成模型筛选、高质量数据生成。其主要方法流程图如图1所示,首先将多个来源收集到的代码数据集进行预处理;然后利用小批量数据和共识投票机制,对众多候选模型进行投票与筛选,组成最佳生成模型池;最后,使用最佳生成模型池,通过TriCogVuln-LLM方法处理大批量原始代码,生成最终的HandPick软件缺陷预测数据集。

2.1 原始多编程语言数据集

本文收集的原始多编程语言代码数据集包含Java、C/C++、Python和JavaScript四种主流编程语言代码样本。该数据集同时包含了原始代码source_code及其对应的修复后代码fix_code。数据集详细信息见表1。由于数据质量存在噪声和不一致性,为确保后续数据生成的质量,本文对数据集进行了必要的数据预处理,具体原始数据集构建和预处理在第3.1.2节中详述。

2.2 TriCogVuln-LLM方法

传统软件缺陷预测方法将缺陷预测任务视为一个分类任务,忽略了缺陷的具体类型、位置信息和缺陷原因。为了提高软件缺陷预测行为的可解释性,本文提出了TriCogVuln-LLM方法。该方法是上

下文工程思想在软件缺陷分析任务中的具体应用,在“CWE缺陷预测→缺陷修复建议生成”的基础上,引入了代码功能描述生成这一关键步骤。这一方法的核心思想是让LLMs充分理解代码上下文语义和实现功能,使其更准确地判断代码中是否存在缺陷。这种渐进式的任务分解方法,为后续软件缺陷分析带来了丰富的上下文感知,在提升预测性能与增强数据可解释性方面展现出双重优势。因此TriCogVuln-LLM工作原理为,先根据原始代码生成对应的功能描述,接着将功能描述和原始代码一同进行CWE缺陷预测,再将得到的预测结果与其原始代码和功能描述结合,生成对应的缺陷修复建议,具体系统提示词示例及工作流程如图2所示。

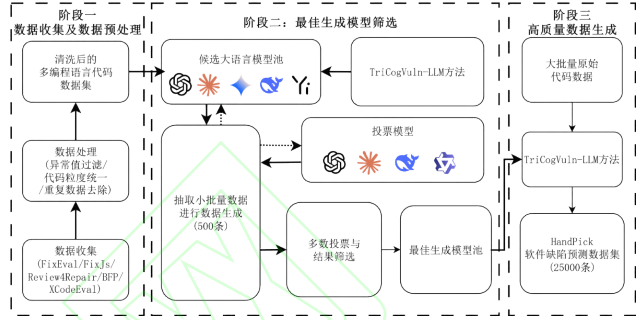


图1 数据集构建主要方法流程

Fig. 1 Flowchart of the Main Methods for Dataset Construction

表1 实验使用的数据集

Tab. 1 Datasets used in the experiment

编程语言类型	数据集名称	数据量	原任务类型
Java	FixEval	43 000	BugFix
	Review4Repair	59 172	BugFix
	BFP	1 190 331	Program Repair
	XCodeEval	574 448	Program Repair
C/C++	XCodeEval	3 409 220	Program Repair
Python	XCodeEval	461 356	Program Repair
JavaScript	FixJs	55 551	BugFix

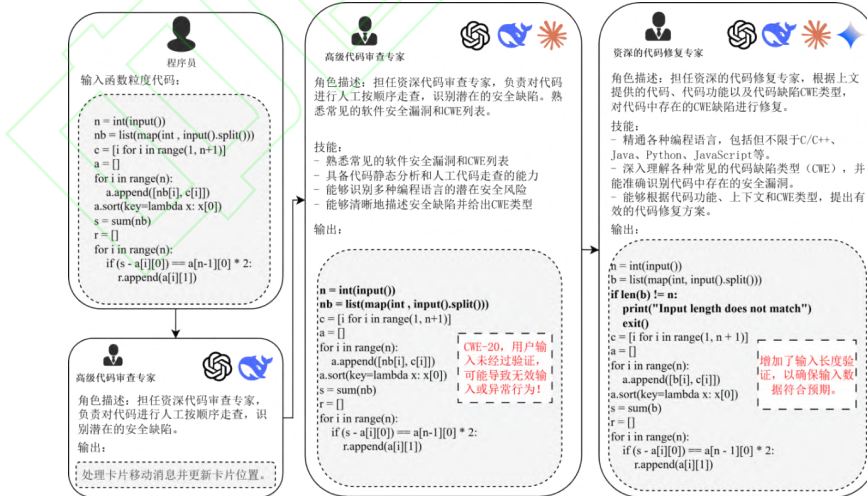


图2 TriCogVuln-LLM工作原理示意图

Fig. 2 Schematic Diagram of the TriCogVuln-LLM Operating Principle

2.2.1 功能描述生成

在TriCogVuln-LLM方法的第一个阶段,模型需要为指定的代码生成其功能摘要。对代码功能的正确理解是软件缺陷预测的前提,代码中是否存在缺陷,取决于它是否偏离了代码的预期功能。该阶段的系统提示词(System Prompt)指示模型扮演一个高级代码审查专家,并要求以不超过30个字的自然语言描述代码的功能。在方法的第一个阶段,让大模型显式地生成功能描述,为后续缺陷判断提供受

约束且具上下文感知的操作语境,避免大模型在功能未知时对软件缺陷类型进行开放式推断。

缺陷预测分析表明,在未知功能条件下的预测范围(ϕ_1)与已知功能的预测范围(ϕ_2)需满足:

$$\phi_2 \subseteq \phi_1 \text{ 且 } \phi_2 \neq \emptyset \quad (1)$$

因此,代码的功能性描述应提供给预测器,引导其在“特定范围”内识别缺陷。在当前公开的数据集中,由于不存在数据集对应代码

片段的功能描述,所以希望LLMs根据当前代码先生成代码功能描述。生成的代码功能描述作为后续任务上下文工程的一部分,为缺陷预测指定了一个替代的预测范围 ϕ_2' ,其中 $\phi_2' \approx \phi_2$ 。这种有针对性的检测确保了修复后的代码符合原始功能需求,并与现实世界的工程实践范式保持一致。

2.2.2 CWE缺陷预测

在功能描述生成后, TriCogVuln-LLM方法在替代的预测范围 ϕ_2' 内启动缺陷预测。在此阶段,系统提示为LLMs分配角色扮演、可用技术和操作注意事项。用户提示提供详细的指令,引入结构化的领域知识,并指定模型所需的输出格式。同时,将收集到的一线开发人员关于软件测试和代码审查的工作经验作为实践知识,与先前的测试经验相结合,共同构成了系统提示。此外,经过对CWE-TOP25排名^[28](2019–2024年)的变动趋势进行深入分析,发现在6年期间部分排名发生了较大变动,但外部输入校验、访问控制问题、注入问题等是过去6年存在的主要安全问题,并会在今后的一段时间持续,所以筛选出10种排名趋势更加稳定的缺陷类型,将其定义为本文的CWE-TOP10进行数据集构建,CWE-TOP10排名详见表2。

该阶段提示词主要包含以下3个关键组成部分:1)角色设定:要求模型扮演一名熟悉多种静态分析工具和安全编码规范的代码审查专家。2)知识注入:为引导模型关注高频、高危缺陷,将本文CWE-TOP10的特征模式作为先验知识编码进提示词中。3)用户指令:明确指示模型需识别代码中是否存在本文CWE-TOP10中的缺陷,如果存在,则必须以严格的JSON格式输出。在CWE缺陷预测的输出内容中包括了缺陷的CWE ID、缺陷所在的代码域、缺陷形成原因及可能造成影响的缺陷描述,最大程度地提升缺陷预测行为的可解释性,以提高其被接受的可能。

2.2.3 缺陷修复建议生成

第三阶段缺陷修复建议生成的目标是为已识别的缺陷生成修复后的代码。本节复用了第二阶段的提示词,并对其进行适应性调整,将任务重心从软件缺陷预测转向缺陷代码修正生成。本阶段的输出内容,不仅包括了缺陷修复后的代码,还包含对于修复后代码的解释,增强开发人员对缺陷修复状况的掌控。

2.3 模型池与共识投票机制

为提升生成数据的多样性并规避单一模型的系统性偏差,本文

提出使用模型池(Model Pool)的方法来进行自动化的数据生成,旨在通过增强训练数据来源的多样性,来缓解分布同质性引发的偏见。本文选择了五个业界顶尖的闭源大语言模型构成候选模型池,抽取小批量的原始数据集,使用TriCogVuln-LLM方法生成数据,目的是在大规模数据生成前对候选模型进行验证。

表2 CWE-TOP10排名

Tab. 2 CWE-TOP10 Ranking

编号	CWE ID	CWE描述	2019	2020	2021	2022	2023	2024
1	CWE-787	越界写入	12	2	1	1	1	2
2	CWE-79	Web页面生成时对输入的转义处理不恰当	2	1	2	2	2	1
3	CWE-89	SQL注入问题	6	6	6	3	3	3
4	CWE-416	内存不安全,释放后使用	7	8	7	7	4	8
5	CWE-78	OS命令注入	11	10	5	6	5	7
6	CWE-20	用户输入未经过验证	3	3	4	4	6	12
7	CWE-125	越界读取	5	4	3	5	7	6
8	CWE-22	文件处理;对路径名的限制不恰当	10	12	8	8	8	5
9	CWE-352	数据真实性验证不足;跨站请求伪造	9	9	9	9	9	4
10	CWE-434	危险类型文件的无限制上传	16	15	10	10	10	10

在获取小规模数据生成结果后,本研究提出了一套新颖的共识投票机制^[29],其设计原则遵循社会学中的“群体智慧”(Wisdom of the Crowd)理论^[30]。该理论指出群体的集体决策通常优于个体,但其有效性极易受到群体中少数“噪声源”的干扰。因此,该机制还借鉴了统计学中的异常检测(Anomaly Detection)思想^[31],将关注点从采纳多数转向识别异常。

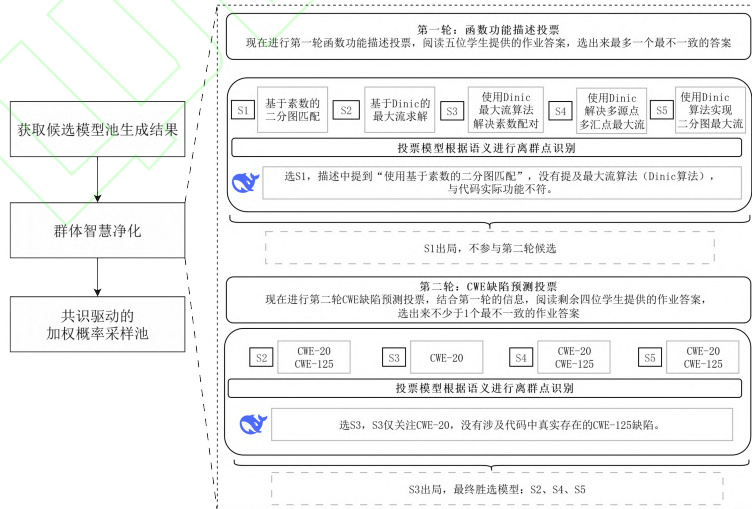


图3 共识投票机制示意图

Fig. 3 Schematic of the Consensus Voting Mechanism

在缺少真实标签(Ground Truth)的情况下,将五个候选模型视为一个待评估的群体,并利用LLMs作为一种先进的语义异常检测器,识别并移除极端的离群点(候选模型)^[32],完成对群体的净化。为确保投票的公正性并克服模型固有的自我偏好,本研究引入一个独立的投票模型进行辅助验证。在移除了最大的不确定性因素后,再对

剩余的“净化群体”应用群体智慧。这个过程并非简单地采取赢家通吃策略,而是根据它们的投票比例构建一个加权概率采样池。例如,一个3:1的意见分布会形成一个按75%/25%概率进行采样的模型池。这种“先净化,后共识”的策略,遵循奥卡姆剃刀原则,既通过排除极端噪声保证了结果的鲁棒性,又通过加权采样保留了群体内部

的健康多样性。这种多样性至关重要,它允许最终的生成数据能蕴含真实问题存在的模糊性或多种合理解释等复杂特征,而不是强行将其简化为单一绝对的答案。共识投票机制示意图如图3所示(其中右侧部分表示投票模型对五个不同候选模型的“净化”过程),本文选取了预处理后的一条代码数据对共识投票机制进行举例说明,首先第一轮任务为函数功能描述投票,根据候选模型生成答案的语义进行离群点识别,淘汰离群点后其余模型进入第二轮评选;第二轮为CWE缺陷预测投票,根据预测的CWE类型、缺陷原因等可解释性信息再进行离群点识别,并结合第一轮结果,选出当前输入下的最佳生成模型池。在所有待投票数据进行以上两轮投票后,按照整体投票结果的比例来组织最佳生成模型池,共识投票过程伪代码实现详见算法1。

算法1 共识投票机制算法。

输入 由候选模型组成的模型池 M_{cand} , 原始数据 D_{raw} , 投票模型 M_{vote} ;
输出 最佳生成模型池 M_{pool} , 缺陷预测数据集 $D_{HandPick}$
1) 定义 M_{drop} 为投票结果的二维列表, R_{raw} 为随机抽取数据集, D_{vote} 为投票阶段使用数据集

$$Score = \frac{100}{4} (\cos(sample_1, pseudo_1) + Jaccard(sample_2, pseudo_2) + Similarity(sample_3, fix_code)) \quad (2)$$

在式(2)中,令 $sample_i$ 和 $pseudo_i$ 分别代表最佳生成模型池采样和伪标签模型生成的第 i 轮输出结果。由于输出结果是在特定的上下文内生成,已经包含了当前任务所关注的上下文语义,同时输出结果是主要评估对象,而引入上下文则会导致待评估对象过长,进而影响最终的评估效果,因此仅采用输出结果作为评估对象。在功能描述任务中,本文使用嵌入模型将 $sample_1$ 和 $pseudo_1$ 转换为嵌入向量,计算它们的余弦相似度。CWE缺陷预测任务的输出 $sample_2$ 和 $pseudo_2$ 被结构化为一个 $l \times k$ 的二元稀疏矩阵,其中 l 为评估样本数, k 为CWE类型ID。在此基础上计算 Jaccard 相似度分数。对于涉及 fix_code 的缺陷修复评估,本文对原始的 fix_code 和生成数据进行词法分析,并将其相似性度量定义为:

$$Similarity(sample_3, fix_code) = \frac{Repetition(sample_3) + Extra(sample_3)}{\frac{1}{n} \left(\sum repetition(sample_3, fix_code) \right) + \min \left(1, \frac{extra(sample_3, fix_code)}{n} \right)} \quad (3)$$

其中 $repetition()$ 表示缺陷修复的代码匹配率, $extra()$ 表示 LLMs 的额外缺陷修复, n 表示 fix_code 修改的行数。

3 实验

本节将介绍实验设置,描述数据收集方法,详述实现细节,并评估生成数据集的质量。

3.1 实验设置与数据集

3.1.1 候选大语言模型

为构建最佳模型池,本文选定五款在代码处理任务中具备代表性的前沿大语言模型作为候选模型,分别为:GPT-4o、DeepSeek-V3、Claude-3.5-Sonnet、Gemini-1.5-Pro 和 Yi-Lightning。这些模型覆盖了不同的技术架构与训练数据,在多个基准测试中表现出强大的性能,能够处理复杂的编程任务,有助于提升生成数据的多样性。本研究使用 MegaVul 测试集,对包含本文数据集验证的模型在内,共 11 个大语言模型进行了性能评估,具体模型情况如表3所示。本研究所有候选模型通过 API 方式调用,生成过程采用各模型提供商的默认推理参数配置。温度参数 $temperature$ 设置为 1.0,最大生成 token 数 max_tokens 设置为 4096,核采样参数 top_p 设置为 0.9,频率惩罚 $frequency_penalty$ 和存在惩罚 $presence_penalty$ 均设置为 0.0。该参数配置在候选模型评估、共识投票机制验证及大规模数据生成的全部

```
2) for generation in Random_raw:
//使用候选模型池生成小批量结果用于共识投票机制
3) D_vote=TriCogVuln(generation, M_cand)
4) for item in D_vote: //进入共识投票的净化投票阶段
5) dropModel_A=firstStage(item, M_cand, M_vote)
6) dropModel_B=secondStage(dropModel_A, item, M_cand, M_vote)
7) M_drop.append(dropModel_A, dropModel_B)
8) M_pool=analysis(M_drop)
//根据投票结果得到最佳生成模型池
9) D_handpick= TriCogVuln(sample(M_pool), D_raw)
//通过对模型池采样,进行共识驱动的数据生成
```

2.4 评估方案

因原始数据集仅包含了原始代码 $source_code$ 及其对应的修复后代码 fix_code , 缺乏如代码功能描述等可用的真实标签,为解决这一问题,本文设计了一种双策略的数据集质量评估方法:1) 将共识投票阶段最优表现者的输出作为伪标签,2) 与原始数据集的 fix_code 实例进行整合。为满足 3 个不同类型的生成任务的评估需求,本文设计了多维度度量框架:

实验阶段保持一致,以确保实验结果的可比性与可重复性。

3.1.2 原始数据集构建与预处理

本研究使用的原始数据集是从多个公开的软件缺陷识别和 bug 修复数据集中汇集而成^[33-37],涵盖了 Java、C/C++、Python 和 JavaScript 四种主流编程语言。为消除原始数据中存在的噪声与不一致性,本研究设计了如下预处理流程:

- 1) 异常值过滤:剔除过长或过短的代码段,并基于规则验证剩余代码的完整性。
 - 2) 代码粒度统一:采用基于正则表达式的模式匹配,将代码统一提取并规范化为函数或类的粒度。
 - 3) 重复数据去除:移除重复的数据条目以确保唯一性。
- 经过预处理,最终获得一个结构统一、内容规范的原始数据集。

表3 实验所使用模型

模型名称	模型类别	调用方式	参数规模
GPT-4o	闭源模型	API 调用	-
Gemini-1.5-pro	闭源模型	API 调用	-
Claude-3.5-Sonnet	闭源模型	API 调用	-
Qwen2.5-14B-Instruct	开源模型	API 调用	14B
Qwen2.5-14B-CodeXGLUE	微调模型	API 调用	14B
Qwen2.5-14B-Draper VDISC	微调模型	API 调用	14B
Qwen2.5-14B-ReposVul	微调模型	API 调用	14B
DeepSeek-V3	开源模型	API 调用	671B
Qwen2.5-72B-Instruct	开源模型	API 调用	72B
Qwen2.5-Coder-32B-Instruct	开源模型	API 调用	32B
Qwen3-32B	开源模型	API 调用	32B
GLM-4-32B-0414	开源模型	API 调用	32B
Qwen2.5-14B-HandPick	微调模型	API 调用	14B

3.2 实验设计与流程

本实验流程分为两个主要阶段:1) 候选模型评估阶段:随机抽取原始数据集中 500 条样本,使用共识投票机制,系统性地评估五款候选模型的性能,并确定用于大规模数据生成的最佳生成模型池。2) 大规模数据生成阶段:利用筛选出的最佳生成模型池,对 25000 条数据进行缺陷预测的数据生成,最终构建 HandPick 数据集。

最终数据集包含以下标签:原始代码、原始代码对应的修复后的代码、功能描述、缺陷预测以及缺陷预测对应的代码修复建议。其中,缺陷预测及其对应的代码修复建议,是软件缺陷预测关注的主要内容,也是 HandPick 数据集与其他数据集相比的新颖之处。

3.3 候选模型评估

本节将对2.3节所述共识投票机制的结果进行分析,筛选出性能可靠且兼具成本效益的模型,以构建最终用于大规模生成数据集的最佳生成模型池。由于投票结果缺少真实标签,因此候选模型评估主要从投票结果本身的数据特征进行分析。

3.3.1 第一轮投票结果分析

第一轮投票模型的对比将侧重于方差指标。为控制实验成本,初始阶段选用DeepSeek-V3与Yi-Lightning两个模型作为投票模型,对全部五个候选模型进行投票。实验结果如图4表示,该图表明,Yi-Lightning作为投票模型时,其投票结果的方差较高,且在投票中频繁将自身淘汰,一致性较差。相比之下,用DeepSeek-V3的投票结果方差较低,表现出更好的一致性。因此,后续分析将主要采信DeepSeek-V3的投票结果,并淘汰Yi-Lightning。

3.3.2 第二轮投票结果分析

为进一步验证DeepSeek-V3作为投票模型的可靠性,选用在第一轮中被普遍认为表现最优的GPT-4o进行第二轮的投票。通过对比GPT-4o的投票分布,来验证DeepSeek-V3的结果是否合理的。如图4所示,综合两轮投票结果,对各候选模型的性能进行分析,结果如下:

GPT-4o:在两项任务中被所有投票模型评为表现最佳,但投票阶段的稳定性不如DeepSeek-V3。

DeepSeek-V3:综合性能仅次于GPT-4o,且表现出极低方差,兼具了性能与计算成本,是最具竞争力的模型。

Claude-3.5-Sonnet:在函数功能描述任务中表现不佳,但在CWE缺陷预测任务上具有独特的角度,其预测结果可以作为对GPT-4o的补充。

Gemini-1.5-Pro与Yi-Lightning:在两项任务中均被多个投票模型频繁投出,综合性能相对较弱。

为进一步验证上述筛选结论的鲁棒性,本文引入Qwen-Max作为一个候选模型分布外的投票模型,对前两轮投票中争议较大的样本进行独立投票。投票结果与GPT-4o和DeepSeek-V3的投票结论高度一致,从而增强了模型筛选结果的可信度。

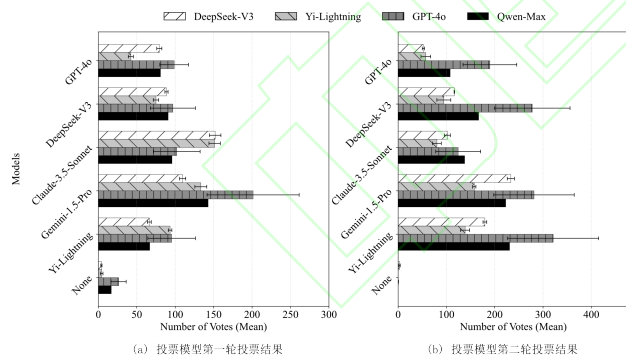


图4 投票模型在小规模数据集上的验证结果

Fig. 4 The validation results of the voting model on a small-scale dataset

3.3.3 模型行为分析

为了确保投票结果的可靠性,本文从模型行为的角度,对投票结果进行了进一步的分析。由于共识投票机制建立在理想的、无歧义的数据之上,因此需要对实验过程中可能出现的数据异常行为及模型幻觉导致的异常行为进行讨论。经过对实验结果的分析,投票模型对上述两种行为表现为:

数据异常行为,具体表现为:

1) 候选模型的结果高度一致,此时投票模型就会将描述性差的候选模型投出,如描述不准确、不简洁等。

2) 候选模型的结果高度不一致,此时投票模型会细致分析各候选模型给出的结果,结合上下文语义进行对比分析,选出描述最合理的一个或两个候选模型,从而淘汰其他的候选模型,如描述不完整、

不正确等。

模型幻觉行为,通常分为忠实性幻觉和事实性幻觉,具体表现为:

1) 忠实性幻觉指的是模型在指令遵循方面的表现,而有效投票结果的数目等于参与投票数据的数目,因此无忠实性幻觉的问题。

2) 事实性幻觉指的是模型在任务处理中出现虚构事实等行为,当前的大语言模型技术无法完全回避此类型幻觉,同时由于LLMs进行语言生成的行为包含随机采样过程,因此事实性幻觉具有随机性。但是投票任务通过大量数据而非单条数据来进行,对于仅有5个候选模型而言,对不同行为的多次采样能够覆盖幻觉的随机性。此外,候选模型的软件缺陷预测能力在共识投票机制上对应的是投票趋势,是一种较为粗糙的任务,尽管可能存在事实性幻觉,但通过多个投票模型的对比交叉验证,可以确定真正合适的投票模型,进而能够保证最佳生成模型池的可靠性。

3.3.4 最佳生成模型池构建策略

基于上述分析,并综合考虑性能、多样性、速度与成本效益,构建了不同阶段数据生成的差异化的模型池:功能描述模块选用性能最优的GPT-4o和性价比最优的DeepSeek-V3,采样比例设为1:2。为增强数据分布多样性,CWE缺陷预测在GPT-4o和DeepSeek-V3的基础上,加入在该任务中表现较好的Claude-3.5-Sonnet,采样比例为4:3:3。缺陷修复建议生成阶段依赖于前两步的输出,且更加侧重生成代码的质量,由于本研究选取的候选模型均具备优秀的代码生成能力,因此本阶段模型池由GPT-4o、DeepSeek-V3、Claude-3.5-Sonnet、Gemini-1.5-Pro组成,采样比例为1:1:1:1。

3.4 生成数据集质量评估

经过对3.1节中的候选模型评估后,最终确定了用于大规模自动化数据生成的模型池,共生成了25000条数据。不同编程语言的CWE-TOP10分布热力图如图5所示。随后,采用第2.4节评估方案对数据集质量进行了评估。在整个投票过程中,GPT-4o的性能始终优于其他模型。因此,在功能描述和CWE缺陷预测任务中,使用GPT-4o的输出作为参考的“伪标签”。此外,整合了原始数据集集中的fix_code进行第三轮评分,以确保评估的全面性。从生成的数据集中选取了500条进行质量评估。

在功能描述任务中,利用BGE M3-Embedding^[38]嵌入模型处理来自GPT-4o直接生成的数据和最佳生成模型池生成的数据。然后,计算了500对数据的余弦相似度,得到平均相似度得分 $\cos(sample_1, pseudo_1) = 0.74$ 。

在CWE缺陷预测任务中,从GPT-4o和模型池的输出中提取了CWE类型编号。遵循第2.4节的评估方法,统计了每种CWE缺陷类型的出现次数。尽管在上下文工程中仅给出了本文定义的CWE-TOP10的信息,但模型自身的世界知识中已经包含CWE相关知识,因此在GPT-4o和模型池生成的数据包含了更多的CWE类型。在抽取的500条数据中,出现了34种不同的CWE类型,其中18种类型是GPT-4o和模型池共有的。

此过程产生了两个形状为(500, 50)的二元稀疏矩阵。通过计算这两个矩阵之间的Jaccard相似度,得分为 $Jaccard(sample_2, pseudo_2) = 0.54$ 。

在缺陷修复建议生成任务中,将LLMs的修复输出在代码行的粒度上,与原始数据集集中的fix_code以及与原始代码的差异进行比较,计算了行内代码重复率。GPT-4o的代码匹配率为 $Repetition(pseudo_3) = 0.6062$ 、最佳生成模型池的代码匹配率为 $Repetition(sample_3) = 0.6190$ 。这表明,与原始数据集集中的真实标签fix_code相比,本研究提出的数据集构建方法表现优于GPT-4o。最佳生成模型池在额外修复率的得分为 $Extra(sample_3) = 1$ 。最终的“缺陷修复建议”任务得分为 $Similarity(sample_3, fix_code) = 1.62$ 。

将以上三部分得分代入公式(2)中,可得 $Score = 72.5$,即HandPick数据集的整体质量评估得分为72.5。当前的评估分数并不能完全代表最终数据集得分,主要原因如下:1) 本研究使用的原始数

数据集均为当下主流且较新的数据集,仍处于不断更新状态,但数据集的更新是进行增量更新,因此SDK老版本始终存在,后续SDK版本的更新已解决了老版本存在的历史问题,导致LLMs不再预测它们。2)一些缺陷与功能需求有关,原始数据集本身缺乏必要的数据(如功能描述等)。3)CWE缺陷预测任务得分较低,这与其固有的挑战性有关,如没有提供所有的CWE信息,且GPT-4o在投票过程中表现出比DeepSeek-V3高的方差,这表明GPT-4o的输出可能不适合作为“伪标签”。

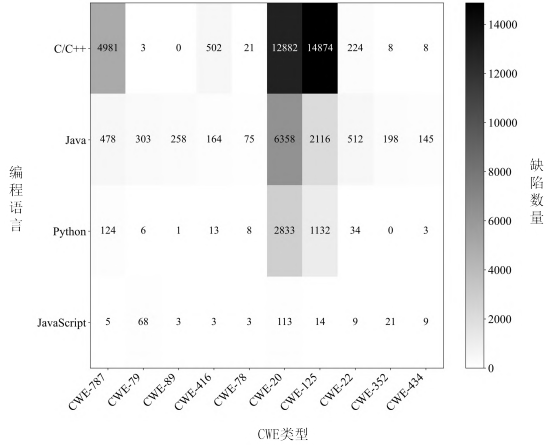


图5 不同编程语言的CWE-TOP10分布热力图

Fig. 5 Heatmap of CWE-TOP10 Distribution Across Different Programming Languages

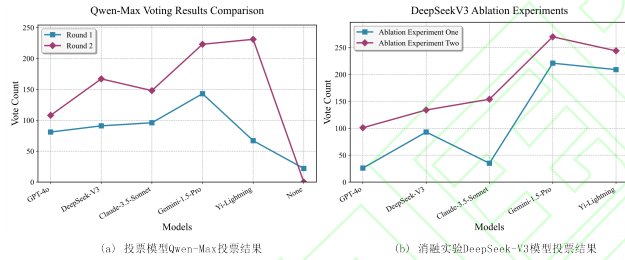


图6 消融实验结果

Fig. 6 Ablation Study Results

3.5 消融实验及结果分析

鉴于选定合适投票模型是评估候选模型关键环节,本节进行了广泛的消融实验并分析了结果。

3.5.1 投票机制公平性验证

投票模型源于候选模型池,这引发了一个潜在的公平性问题:模型可能因其固有的知识分布或句法结构偏好,而在评估中倾向于自身生成的内容,从而产生“自我偏袒”现象。尽管初步观察到Yi-Lightning模型未表现出明显的偏袒行为,但当模型同时扮演“选手”与“裁判”双重角色时,系统性偏见的风险依然存在。

为探究此风险的实际影响,本文引入一个候选模型分布外的投票模型Qwen-Max执行相同的投票任务。图6(a)的实验结果表明,候选模型分布外的投票结论与内部模型的结论高度一致。这说明,尽管理论上存在偏袒风险,但经过消融实验,证明本研究的实验框架并未产生偏袒现象。

3.5.2 一阶段投票对二阶段投票的影响

在本文设计的两阶段投票流程中,第一阶段功能描述生成的筛选结果将直接影响进入第二阶段CWE缺陷预测的模型范围。考虑到功能描述任务的相对简单性,一个核心问题随之产生:第一阶段的筛选是否确实有必要?它是否会对最终的模型评估与选择产生关键影响?

为了探究此问题,本文关注到一个具体的、足以体现该影响的现

象:在标准的投票流程中,Claude-3.5-Sonnet模型在第二阶段投票中淘汰率极低。这一现象引出了两种截然相反的解释:其一,这是一种由第一阶段筛选造成的性能不佳的实例被提前剔除,从而美化了其在第二阶段的表现;其二,该模型在CWE缺陷预测任务上确实具备独特的竞争力,与第一阶段的筛选无关。

本小节由此设计了消融实验方案:完全移除第一阶段投票,使用投票过程表现较好的Deepseek-V3再次投票,让所有候选模型直接进入第二阶段进行评估。图6(b)所示的消融实验结果有力地支持了Claude-3.5-Sonnet在CWE缺陷预测任务上确实具备独特优势这个观点。

3.6 下游任务性能验证

3.6.1 实验设置

为验证HandPick数据集的实用价值,本文使用其对Qwen2.5-14B-Instruct模型进行微调,并在一个独立的公开测试集MegaVul上评估其软件缺陷预测能力。

MegaVul是一个多语言、多来源的软件缺陷集合,整合了多个主流数据集(如Vul4J、Devign、CodeXGLUE-Vul、Python-Bug-Fix等),涵盖Java、C/C++、Python等常见编程语言,并对样本中的缺陷标签进行了人工标注与校验,广泛用于漏洞检测与缺陷定位任务的评估场景。

本文从公开的MegaVul数据集中,构建了一个包含1000个代码样本的测试集。该测试集包含861个属于CWE-TOP10范围内的缺陷样本和139个无缺陷样本,以模拟真实场景下的缺陷分布。实验将微调后的模型Qwen2.5-14B-HandPick与GPT-4o、Claude-3.5-Sonnet、Gemini-1.5-Pro、Qwen2.5-14B-Instruct、DeepSeek-V3、Qwen2.5-72B-Instruct、Qwen2.5-Coder-32B-Instruct、Qwen3-32B、GLM-4-32B-0414模型进行性能比较。当前人工智能在软件工程相关领域的研究集中在代码生成任务,而软件缺陷预测领域,尤其是缺陷预测数据集的研究工作,进展较为滞后。为全面验证HandPick数据集的有效性,本研究选取了三个具有代表性的缺陷预测数据集进行对比实验。CodeXGLUE是专为代码理解和生成研究设计的综合性机器学习基准数据集,涵盖多种代码相关任务。Draper VDISC^[39]数据集由美国Draper实验室构建,包含从开源项目中提取的大规模C/C++函数级缺陷样本。ReposVul^[40]是基于真实代码仓库提交历史构建的多语言软件缺陷数据集,保留了完整的代码上下文信息。本研究使用这三个数据集分别微调得到Qwen2.5-14B-CodeXGLUE、Qwen2.5-14B-Draper VDISC和Qwen2.5-14B-ReposVul模型,所有模型与本文微调的Qwen2.5-14B-HandPick模型采用完全相同的微调超参数,仅训练数据来源不同。此项对比实验消融了训练框架的影响,独立地验证了HandPick数据集相较于传统数据集的优越性。所有模型均采用相同的、本研究设计的TriCogViln-LLM方法进行零样本(zero-shot)推理。

表4 各个模型在MegaVul测试集的表现

Tab. 4 Performance of various models on the MegaVul test set

模型	Precision	Recall	F1-Score	Accuracy
GPT-4o	0.7181	0.4116	0.5233	0.3544
Gemini-1.5-Pro	0.5900	0.2323	0.3333	0.2000
Claude-3.5-Sonnet	0.7832	0.5878	0.6716	0.5055
Qwen2.5-14B-Instruct	0.5088	0.1672	0.2517	0.1440
Qwen2.5-14B-CodeXGLUE	0.5901	0.1103	0.1859	0.1680
Qwen2.5-14B-Draper VDISC	0.4317	0.2711	0.3330	0.2910
Qwen2.5-14B-ReposVul	0.2640	0.3269	0.2921	0.2680
DeepSeek-V3	0.6091	0.2494	0.3539	0.2150
Qwen2.5-72B-Instruct	0.7002	0.3785	0.4914	0.4100
Qwen2.5-Coder-32B-Instruct	0.6345	0.1137	0.1929	0.2530
Qwen3-32B	0.6387	0.1227	0.2058	0.2600
GLM-4-32B-0414	0.5527	0.5353	0.5439	0.4650
Qwen2.5-14B-HandPick	0.7017	0.3798	0.4928	0.3270

3.6.2 评测指标定义

本文采用标准的分类任务指标评估模型性能。混淆矩阵的四个基本组成部分定义如下:真阳性(*TP*)指样本实际上存在漏洞,模型正确地预测其为有漏洞,并且预测的CWE ID中至少有一个与样本的真实CWE ID相匹配。假阳性(*FP*)指样本实际上无漏洞,但模型错误地预测其为有漏洞。假阴性(*FN*)指样本实际上存在漏洞,但模型未能识别出其任何一个真实的CWE类型,将其归类为无漏洞或预测了完全错误的CWE类型。真阴性(*TN*)指样本实际上无漏洞,且模型正确地识别出这一点。

基于这些组成部分,计算了标准的分类性能指标,包括准确率(*Accuracy*)、精确率(*Precision*)、召回率(*Recall*)和F1分数(*F1 - score*)。公式如下:

$$Accuracy = \frac{TP + TN}{TP + FP + TN + FN} \quad (4)$$

$$Precision = \frac{TP}{TP + FP} \quad (5)$$

$$Recall = \frac{TP}{TP + FN} \quad (6)$$

$$F1 - score = 2 \times \frac{Precision \times Recall}{Precision + Recall} \quad (7)$$

3.6.3 实验结果与分析

各模型在MegaVul测试集上的详细性能数据如表4所示。从实验结果中,可以得出以下结论:

1) HandPick数据集的有效性得到验证:与原始模型相比,使用HandPick微调后的Qwen2.5-14B-HandPick性能得到了全面且显著的提升。其中,F1分数从0.2517提升至0.4928,提高了24.11个百分点,精确率、召回率和准确率也分别提升了19.29、21.26和18.30个百分点。这显著地证明了本文构建的数据集能够有效增强模型在缺陷预测任务上的能力,特别是在减少漏报方面的有效性。

2) 与主流模型的性能对比:微调后的模型在综合性能上,显著优于DeepSeek-V3和Gemini-1.5-Pro。尽管与当前顶尖的闭源模型(如Claude-3.5-Sonnet)相比仍存在差距,但微调得到的Qwen2.5-14B-HandPick,以更少的参数量达到了堪比GPT-4o的优秀表现,证明了HandPick能够有效提升模型软件缺陷预测能力,适用于轻量化部署。

本文提出的TriCogVuln-LLM方法和共识投票机制,能够构建出具备可解释性的软件缺陷领域专用数据集,相比仅有软件缺陷预测分类的CodeXGLUE数据集,通过引入缺陷预测的可解释性,更有效提升了现有大语言模型在软件缺陷预测这一专业任务的性能。同时,出于工程实际部署使用中的源代码安全、部署成本以及实际推理速度等方面的考虑,通过微调更符合实际部署要求的轻量化、小尺寸模型,在零样本推理设置下,表现出了较好的软件缺陷预测能力,同时能够提供缺陷预测的解释性信息,提高了缺陷预测的结果被接受的可能,更加印证本文方法的有效性。

4 结论

在本研究中,从三个关键维度做出了贡献:

1) 提出了一个多语言、可供大语言模型训练且具备可解释性的代码缺陷预测数据集——HandPick。这一数据集的构建填补了现有软件缺陷数据集中可解释性不足的空白。

2) 设计了TriCogVuln-LLM方法,结合共识投票机制,在数据生成的成本、质量和多样性之间取得了平衡。

3) 为TriCogVuln-LLM方法设计了一套相应的评估方法。在该评估体系下,HandPick数据集取得了72.5分的高分,经下游任务验证表明,微调后的模型在独立公开测试集上的表现显著提升,这凸显了其在不同编程语言的缺陷预测中的有效性。

本研究的核心贡献,不仅在于提供了一个新颖的数据集,更在于它响应了人工智能正向以数据为中心转变的时代呼唤。在软件缺陷预测这一专业领域,缺少高质量、可解释的数据集正是限制LLMs应用深度与广度的核心瓶颈,也严重阻碍了预测结果在实际软件开发

工作中的应用。因此,HandPick数据集不仅是技术上的创新,更重要的是,它们为构筑下一代更可靠、更值得信赖的软件缺陷预测模型,提供了最关键的、以数据为核心的基石。本研究中的局限性来源于仅使用编程语言的原生代码序列,并未考虑使用其他模态的信息来辅助CWE缺陷预测,例如控制流图等。在未来研究中,将进行多模态信息融合,结合控制流图等多模态信息,提高软件缺陷的预测质量,同时 will 使用HandPick数据集微调出的模型投入实际工程使用,并利用生产数据来进行持续学习,进一步提高模型的预测能力。

参考文献 (References)

- [1] Azam M, Nouman M, Gill A R. Comparative Analysis of Machine Learning Technique to Improve Software Defect Prediction: Comparative Analysis of Machine Learning Technique to Improve Software Defect Prediction [J]. KIET Journal of Computing and Information Sciences, 2022, 5(2).
- [2] Tan Z, Anagnostopoulos C, Singer J. OSPtrack: A Labeled Dataset Targeting Simulated Execution of Open-Source Software [C]//2025 IEEE/ACM 22nd International Conference on Mining Software Repositories (MSR). IEEE, 2025: 659-663.
- [3] Mahbub P, Shuvo O, Rahman M M. Defectors: A large, diverse python dataset for defect prediction [C]//2023 IEEE/ACM 20th International Conference on Mining Software Repositories (MSR). IEEE, 2023: 393-397.
- [4] Lu S, Guo D, Ren S, et al. CodeXGLUE: A machine learning benchmark dataset for code understanding and generation [J]. arXiv preprint arXiv:2102.04664, 2021.
- [5] Gezici Geçer B, Kolkusa Tarhan A. Explainable AI Framework for Software Defect Prediction [J]. Journal of Software: Evolution and Process, 2025, 37(4): e70018.
- [6] Wang J, Huang Y, Chen C, et al. Software testing with large language models: Survey, landscape, and vision [J]. IEEE Transactions on Software Engineering, 2024, 50(4): 911-936.
- [7] Zhang Q, Fang C, Xie Y, et al. A survey on large language models for software engineering [J]. arXiv preprint arXiv: 2312.15223, 2023.
- [8] Zhang Z, Chen C, Liu B, et al. Unifying the perspectives of nlp and software engineering: A survey on language models for code [J]. arXiv preprint arXiv:2311.07989, 2023.
- [9] Borandag E. Software fault prediction using an RNN-based deep learning approach and ensemble machine learning techniques [J]. Applied Sciences, 2023, 13(3): 1639.
- [10] Li Z, Li T, Wu Y M, et al. Software defect prediction based on hybrid swarm intelligence and deep learning [J]. Computational Intelligence and Neuroscience, 2021, 2021(1): 4997459.
- [11] Phuong H T M, Ngan D T K, Binh N T. A comparative study of deep learning techniques in software fault prediction [J]. , 2024: 1-5.
- [12] Shin J, Aleithan R, Nam J, et al. Explainable software defect prediction: Are we there yet? [J]. arXiv preprint arXiv: 2111.10901, 2021.
- [13] Nguyen D H, Seo A, Nnamdi N P, et al. False alarm reduction method for weakness static analysis using bert model [J]. Applied Sciences, 2023, 13(6): 3502.
- [14] Ashraf H, Alenezi M, Nadeem M, et al. Security assessment framework for educational ERP systems [J]. International Journal of Electrical and Computer Engineering, 2019, 9(6): 5570.
- [15] Kim D, Jeon S, Kim K, et al. Guide to developing case-based attack scenarios and establishing defense strategies for cybersecurity exercise in ICS environment [J]. The Journal of Supercomputing, 2024, 80(15): 21642-21675.

- [16] Ouyang L, Wu J, Jiang X, et al. Training language models to follow instructions with human feedback [J]. *Advances in neural information processing systems*, 2022, 35: 27730-27744.
- [17] Cheng Y, Shar L K, Zhang T, et al. Llm-enhanced static analysis for precise identification of vulnerable oss versions [J]. *arXiv preprint arXiv:2408.07321*, 2024.
- [18] Fan L, Liu J, Liu Z, et al. Exploring the capabilities of llms for code change related tasks [J]. *ACM Transactions on Software Engineering and Methodology*, 2024.
- [19] Hou X, Zhao Y, Liu Y, et al. Large language models for software engineering: A systematic literature review [J]. *ACM Transactions on Software Engineering and Methodology*, 2024, 33(8): 1-79.
- [20] Kulsum U, Zhu H, Xu B, et al. A case study of llm for automated vulnerability repair: Assessing impact of reasoning and patch validation feedback [C]//*Proceedings of the 1st ACM International Conference on AI-Powered Software*. 2024: 103-111.
- [21] Zhou X, Kim K, Xu B, et al. Out of sight, out of mind: Better automatic vulnerability repair by broadening input ranges and sources [C]//*Proceedings of the IEEE/ACM 46th international conference on software engineering*. 2024: 1-13.
- [22] Zamfirescu-Pereira J D, Wong R Y, Hartmann B, et al. Why Johnny can't prompt: how non-AI experts try (and fail) to design LLM prompts [C]//*Proceedings of the 2023 CHI conference on human factors in computing systems*. 2023: 1-21.
- [23] Wei J, Wang X, Schuurmans D, et al. Chain-of-thought prompting elicits reasoning in large language models [J]. *Advances in neural information processing systems*, 2022, 35: 24824-24837.
- [24] Liu J, Xia C S, Wang Y, et al. Is your code generated by chatgpt really correct? rigorous evaluation of large language models for code generation [J]. *Advances in Neural Information Processing Systems*, 2023, 36: 21558-21572.
- [25] Cheng Z, Wan D, Abueg M, et al. CI-Bench: Benchmarking Contextual Integrity of AI Assistants on Synthetic Data [J]. *arXiv e-prints*, 2024; *arXiv: 2409.13903*.
- [26] Elkins S, Kochmar E, Serban I, et al. How useful are educational questions generated by large language models? [C]//*International Conference on Artificial Intelligence in Education*. Cham: Springer Nature Switzerland, 2023: 536-542.
- [27] He Z, Jiang H, Wang Z, et al. Position Engineering: Boosting Large Language Models through Positional Information Manipulation [C]//*Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. 2024: 7333-7345.
- [28] MITRE. 2023 CWE Top 25 Most Dangerous Software Weaknesses [R/OL]. (2023-06-28) [2025-08-22]. https://cwe.mitre.org/top25/archive/2023/2023_cwe_top25.html.
- [29] Gu J, Jiang X, Shi Z, et al. A survey on llm-as-a-judge [J]. *arXiv preprint arXiv:2411.15594*, 2024.
- [30] Mennis E A. The wisdom of crowds: Why the many are smarter than the few and how collective wisdom shapes business, economies, societies, and nations [J]. *Business Economics*, 2006, 41(4): 63-65.
- [31] Aggarwal C. C. *Outlier Analysis* [M]. Springer, 2017.
- [32] Hawkins D. M. *Identification of outliers* [M]. Springer, 1980.
- [33] Haque M M A, Ahmad W U, Lourentzou I, et al. Fixeval: Execution-based evaluation of program fixes for programming problems [C]//*2023 IEEE/ACM International Workshop on Automated Program Repair (APR)*. IEEE, 2023: 11-18.
- [34] Huq F, Hasan M, Haque M M A, et al. Review4repair: Code review aided automatic program repairing [J]. *Information and Software Technology*, 2022, 143: 106765.
- [35] Tufano M, Watson C, Bavota G, et al. An empirical investigation into learning bug-fixing patches in the wild via neural machine translation [C]//*Proceedings of the 33rd ACM/IEEE international conference on automated software engineering*. 2018: 832-837.
- [36] Khan M A Mss, Bari M S, Do X L, et al. xcodeeval: A large scale multilingual multitask benchmark for code understanding, generation, translation and retrieval [J]. *arXiv preprint arXiv: 2303.03004*, 2023.
- [37] Csuvik V, Vidács L. Fixjs: A dataset of bug-fixing javascript commits [C]//*Proceedings of the 19th International Conference on Mining Software Repositories*. 2022: 712-716.
- [38] Chen J, Xiao S, Zhang P, et al. Bge m3-embedding: Multilingual, multi-functionality, multi-granularity text embeddings through self-knowledge distillation [J]. *arXiv preprint arXiv: 2402.03216*, 2024.
- [39] Russell R, Kim L, Hamilton L, et al. Automated vulnerability detection in source code using deep representation learning [C]//*2018 17th IEEE international conference on machine learning and applications (ICMLA)*. IEEE, 2018: 757-762.
- [40] Wang X, Hu R, Gao C, et al. Reposvul: A repository-level high-quality vulnerability dataset [C]//*Proceedings of the 2024 IEEE/ACM 46th International Conference on Software Engineering: Companion Proceedings*. 2024: 472-483.